

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09 COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations

QUESTIONS: 5

Total Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

I **Duddela Harika / 192324301**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D. Harika

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

Code:

```
import java.util.Scanner;

class Book {

    private int bookId;

    private String title;

    private String author;

    private double price;

    private boolean issued;

    public Book(int id, String t, String a, double p) {

        bookId = id;

        title = t;

        author = a;

        price = p;

        issued = false;

    }

    public int getBookId() {

        return bookId;

    }

    public void issueBook() {

        if (!issued) {

            issued = true;

            System.out.println("Book Issued Successfully");

        } else {
```

```

        System.out.println("Book Already Issued");
    }
}

public void returnBook() {
    if (issued) {
        issued = false;

        System.out.println("Book Returned Successfully");
    } else {
        System.out.println("Book is Already Available");
    }
}

public void display() {
    System.out.println(bookId + " " + title + " " + author + " Rs." + price +
        " " + (issued ? "Issued" : "Available"));
}
}

public class LibraryManagement {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        Book books[] = new Book[10];

        int count = 0, choice, id;

        boolean found;

        while (true) {
            System.out.println("\n1.Add Book");

            System.out.println("2.Search Book");

            System.out.println("3.Issue Book");

            System.out.println("4.Return Book");

            System.out.println("5.Display Books");

```

```
System.out.println("6.Exit");

System.out.print("Enter Choice: ");

choice = sc.nextInt();

switch (choice) {

    case 1:

        System.out.print("Book ID: ");

        id = sc.nextInt();

        sc.nextLine();

        System.out.print("Title: ");

        String title = sc.nextLine();

        System.out.print("Author: ");

        String author = sc.nextLine();

        System.out.print("Price: ");

        double price = sc.nextDouble();

        books[count++] = new Book(id, title, author, price);

        System.out.println("Book Added Successfully");

        break;

    case 2:

        System.out.print("Enter Book ID: ");

        id = sc.nextInt();

        found = false;

        for (int i = 0; i < count; i++) {

            if (books[i].getBookId() == id) {

                books[i].display();

                found = true;

                break;

            }

        }

    }

}
```

```
if (!found)

    System.out.println("Book Not Found");

    break;
```

case 3:

```
System.out.print("Enter Book ID: ");

id = sc.nextInt();

found = false;

for (int i = 0; i < count; i++) {

    if (books[i].getBookId() == id) {

        books[i].issueBook();

        found = true;

        break;

    }

}
```

```
if (!found)

    System.out.println("Book Not Found");

    break;
```

case 4:

```
System.out.print("Enter Book ID: ");

id = sc.nextInt();

found = false;

for (int i = 0; i < count; i++) {

    if (books[i].getBookId() == id) {

        books[i].returnBook();

        found = true;

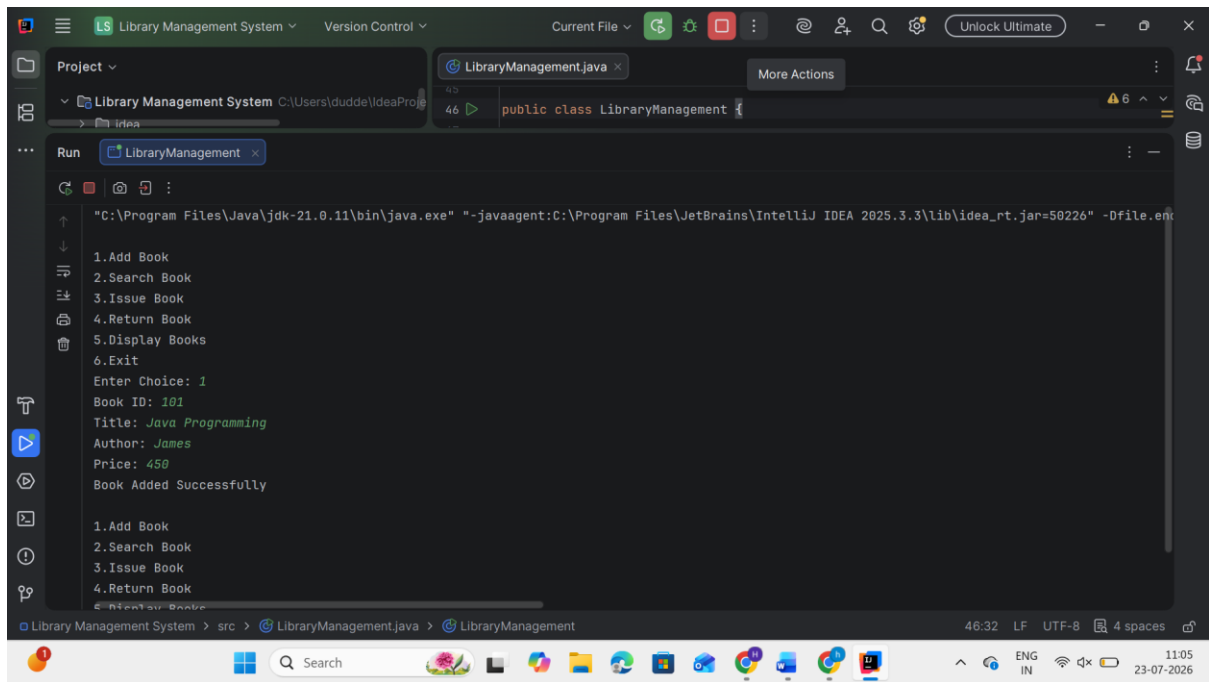
        break;

    }

}
```

```
        if (!found)
            System.out.println("Book Not Found");
        break;
case 5:
    if (count == 0)
        System.out.println("No Books Available");
    else
        for (int i = 0; i < count; i++)
            books[i].display();
        break;
case 6:
    System.exit(0);
default:
    System.out.println("Invalid Choice");
    }
    }
    }
}
```

Output:



2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes Permanent Employee and Contract Employee. Override a method calculate Salary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types.

Code:

```
class Employee {

    String name;

    Employee(String name) {

        this.name = name;

    }

    void calculateSalary() {

        System.out.println("Salary");

    }

}

class PermanentEmployee extends Employee {

    PermanentEmployee(String name) {

        super(name);

    }

}
```

```

    }

    void calculateSalary() {
        System.out.println(name + " Salary = 50000");
    }
}

class ContractEmployee extends Employee {
    ContractEmployee(String name) {
        super(name);
    }

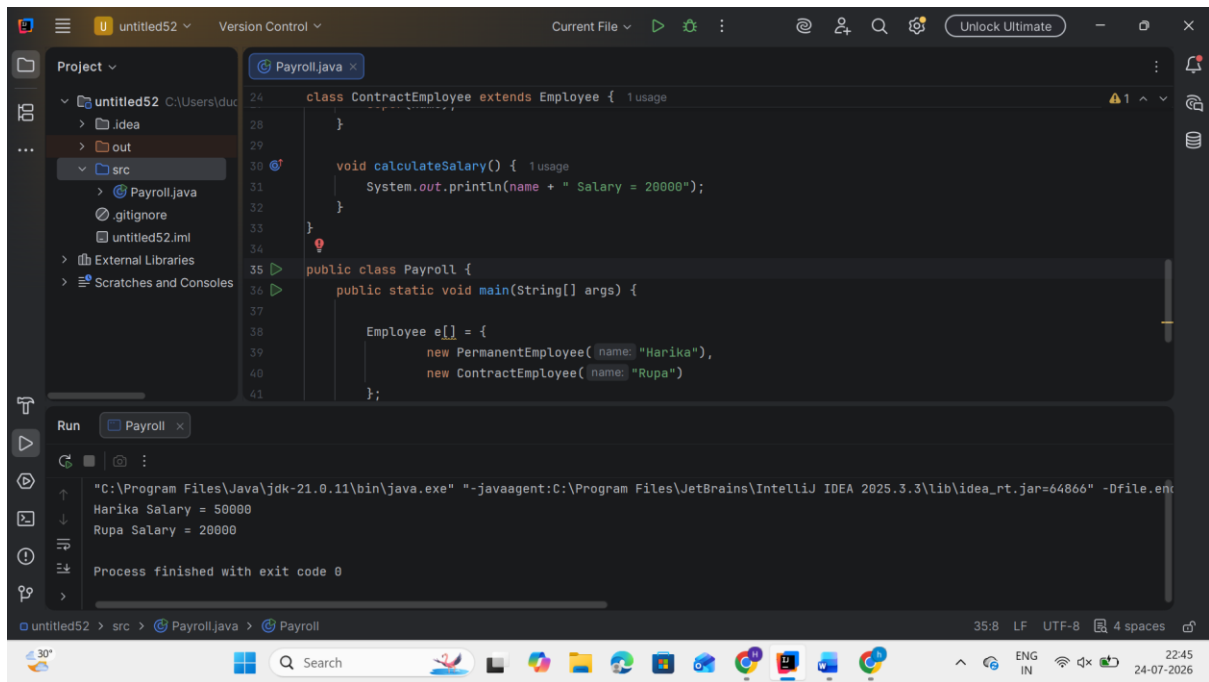
    void calculateSalary() {
        System.out.println(name + " Salary = 20000");
    }
}

public class Payroll {
    public static void main(String[] args) {
        Employee e[] = {
            new PermanentEmployee("Harika"),
            new ContractEmployee("Rupa")
        };

        for (Employee emp : e)
            emp.calculateSalary();
    }
}

```

Output:



3. Vehicle Rental System using Runtime Polymorphism

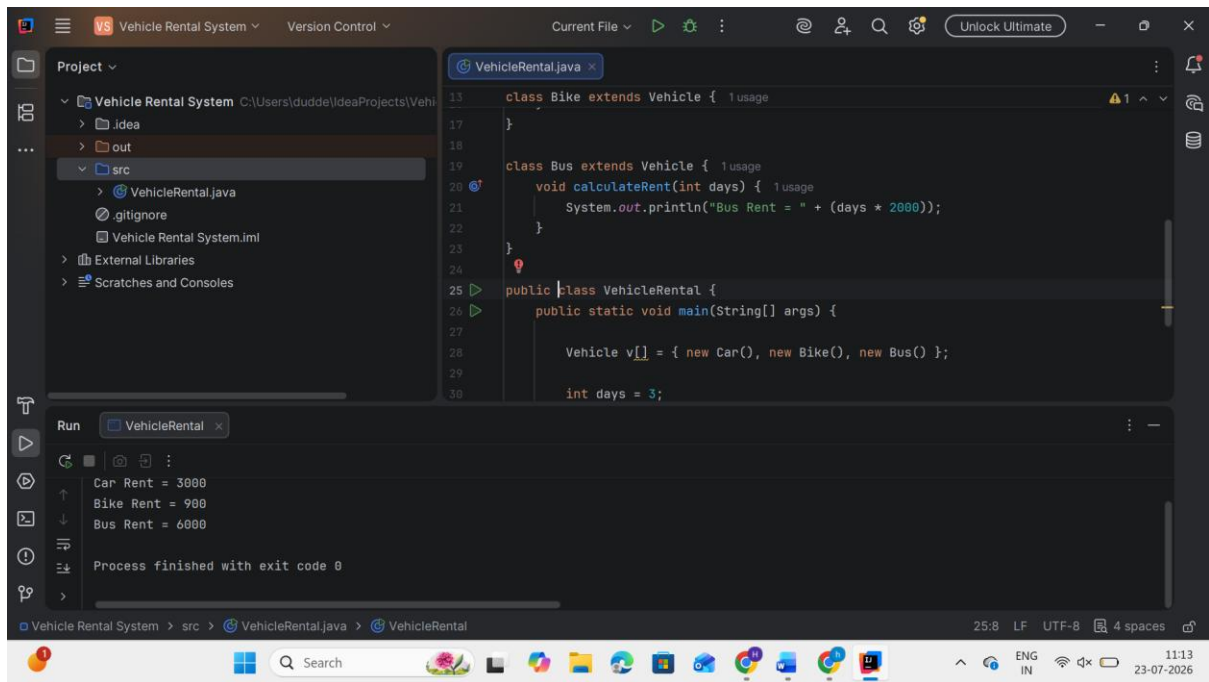
Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories.

Code:

```
class Vehicle {  
  
    void calculateRent(int days) {  
  
        System.out.println("Rent = 0");  
  
    }  
  
}  
  
class Car extends Vehicle {  
  
    void calculateRent(int days) {  
  
        System.out.println("Car Rent = " + (days * 1000));  
  
    }  
  
}  
  
class Bike extends Vehicle {
```

```
void calculateRent(int days) {  
    System.out.println("Bike Rent = " + (days * 300));  
}  
  
class Bus extends Vehicle {  
    void calculateRent(int days) {  
        System.out.println("Bus Rent = " + (days * 2000));  
    }  
}  
  
public class VehicleRental {  
    public static void main(String[] args) {  
        Vehicle v[] = { new Car(), new Bike(), new Bus() };  
        int days = 3;  
        for (Vehicle x : v)  
            x.calculateRent(days);  
    }  
}
```

Output:



4. Bank Account System using Data Hiding

Design a Bank Account class with private fields account Number, holder Name, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions.

Code:

```
import java.util.Scanner;

class BankAccount {

    private int accountNumber;

    private String holderName;

    private double balance;

    BankAccount(int a, String h, double b) {

        accountNumber = a;

        holderName = h;

        balance = b;

    }

    void deposit(double amt) {

        if (amt > 0) {
```

```

        balance += amt;

        System.out.println("Amount Deposited");
    } else

        System.out.println("Invalid Amount");
    }

    void withdraw(double amt) {
        if (amt <= balance) {
            balance -= amt;

            System.out.println("Amount Withdrawn");
        } else

            System.out.println("Insufficient Balance");
        }

    void showBalance() {
        System.out.println("Balance = " + balance);
    }
}

public class BankDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        BankAccount b = new BankAccount(101, "Harika", 5000);

        System.out.print("Enter Deposit Amount: ");

        b.deposit(sc.nextDouble());

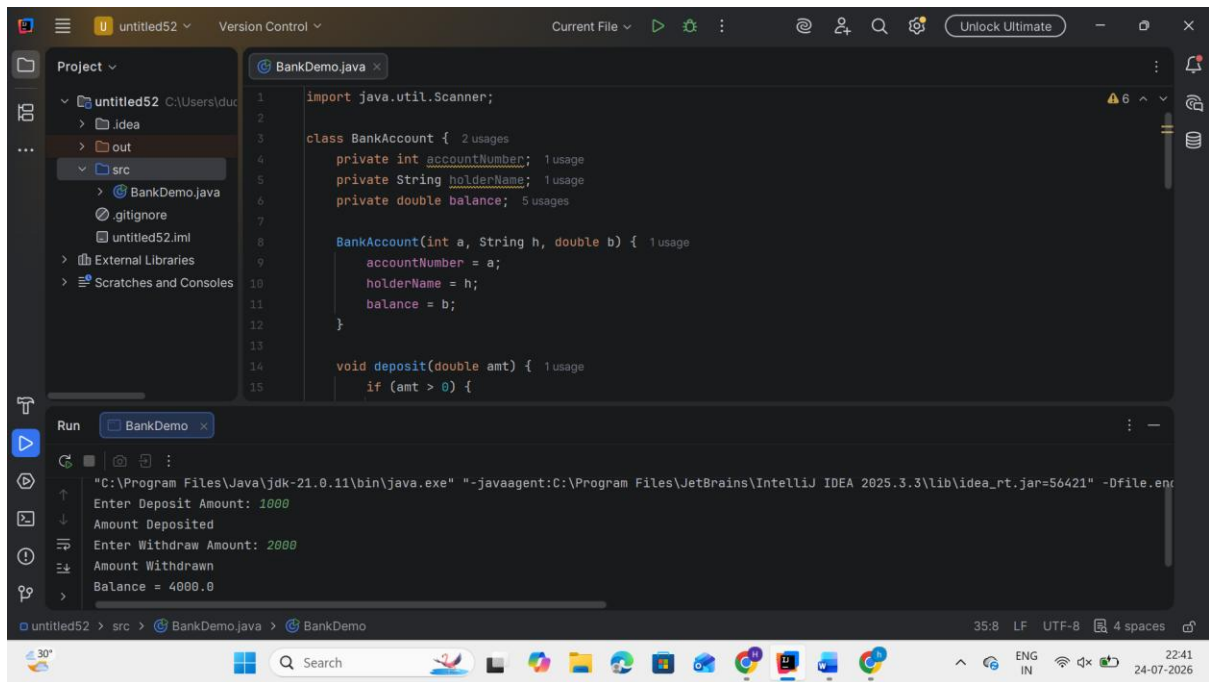
        System.out.print("Enter Withdraw Amount: ");

        b.withdraw(sc.nextDouble());

        b.showBalance();
    }
}

```

Output:



5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

Undergraduate students,

Postgraduate students,

Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version.

Code:

```
import java.util.Scanner;

class Admission {

    void calculateFee() {

        System.out.println("Undergraduate Fee = 50000");

    }

    void calculateFee(int pg) {

        System.out.println("Postgraduate Fee = 70000");

    }

    void calculateFee(double scholarship) {
```

```

        System.out.println("Scholarship Fee = 30000");
    }
}

public class UniversityAdmission {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Admission a = new Admission();
        System.out.println("1. Undergraduate");
        System.out.println("2. Postgraduate");
        System.out.println("3. Scholarship");
        System.out.print("Enter Choice: ");
        int ch = sc.nextInt();
        switch(ch) {
            case 1:
                a.calculateFee();
                break;
            case 2:
                a.calculateFee(1);
                break;
            case 3:
                a.calculateFee(1.0);
                break;
            default:
                System.out.println("Invalid Choice");
        }
    }
}

```

Output:

